

SUBCONSULTAS. Una subconsulta es un comando SELECT dentro de otro comando <pre>SELECT column_name [, column_name] FROM table1 [, table2] WHERE column_name OPERATOR (SELECT column_name [, column_name] FROM table1 [, table2] [WHERE])</pre> Las principales ventajas de subconsultas son: • Permiten consultas estructuradas de forma que es posible aislar cada parte de un comando. • Proporcionan un modo alternativo de realizar operaciones que de otro modo necesitarían joins y uniones complejos.	TIPOS DE SUBCONSULTAS <table border="1"> <tr> <td>Subconsultas ESCALARES</td><td>Devuelven un solo valor, es decir, una fila con una columna de datos. Podemos utilizar subconsultas como parámetros de funciones</td></tr> <tr> <td colspan="2"> SELECT Country.Name FROM Country, City WHERE Country.Code = City.CountryCode AND City.Population = (SELECT MAX(Population) FROM City); </td></tr> <tr> <td>Subconsultas de COLUMNA</td><td>Devuelven una sola columna con una o más filas de datos.</td></tr> <tr> <td colspan="2"> Muestra las lenguas que se hablan en FINLANDIA. SELECT Language FROM CountryLanguage WHERE CountryCode = (SELECT Code FROM Country WHERE Name='Finland'); </td></tr> <tr> <td>Subconsultas de REGISTRO</td><td>Devuelven una única fila con una o más columnas de datos. No es muy habitual.</td></tr> <tr> <td colspan="2"> Ejemplo: Capital de España SELECT name FROM world.city WHERE(id, countrycode) =(SELECT capital, code FROM world.country WHERE name='Spain'); </td></tr> <tr> <td>Subconsultas de tipo TABLA</td><td>Devuelven un resultado con una o más filas que contienen una o más columnas de datos.</td></tr> </table> Las subconsultas de tipo tabla pueden aparecer en 2 contextos: • En la cláusula FROM • Como operando en la parte derecha de un operador IN, NOT IN, EXISTS, NOT EXISTS, o ALL, ANY, SOME anteponiendo uno de los siguientes operadores (=, !=, <>, <, >, <=, >=).	Subconsultas ESCALARES	Devuelven un solo valor, es decir, una fila con una columna de datos. Podemos utilizar subconsultas como parámetros de funciones	SELECT Country.Name FROM Country, City WHERE Country.Code = City.CountryCode AND City.Population = (SELECT MAX(Population) FROM City);		Subconsultas de COLUMNA	Devuelven una sola columna con una o más filas de datos.	Muestra las lenguas que se hablan en FINLANDIA. SELECT Language FROM CountryLanguage WHERE CountryCode = (SELECT Code FROM Country WHERE Name='Finland');		Subconsultas de REGISTRO	Devuelven una única fila con una o más columnas de datos. No es muy habitual.	Ejemplo: Capital de España SELECT name FROM world.city WHERE (id, countrycode) =(SELECT capital, code FROM world.country WHERE name='Spain');		Subconsultas de tipo TABLA	Devuelven un resultado con una o más filas que contienen una o más columnas de datos.
Subconsultas ESCALARES	Devuelven un solo valor, es decir, una fila con una columna de datos. Podemos utilizar subconsultas como parámetros de funciones														
SELECT Country.Name FROM Country, City WHERE Country.Code = City.CountryCode AND City.Population = (SELECT MAX(Population) FROM City);															
Subconsultas de COLUMNA	Devuelven una sola columna con una o más filas de datos.														
Muestra las lenguas que se hablan en FINLANDIA. SELECT Language FROM CountryLanguage WHERE CountryCode = (SELECT Code FROM Country WHERE Name='Finland');															
Subconsultas de REGISTRO	Devuelven una única fila con una o más columnas de datos. No es muy habitual.														
Ejemplo: Capital de España SELECT name FROM world.city WHERE (id, countrycode) =(SELECT capital, code FROM world.country WHERE name='Spain');															
Subconsultas de tipo TABLA	Devuelven un resultado con una o más filas que contienen una o más columnas de datos.														
SUBCONSULTAS CORRELACIONADAS Y NO CORRELACIONADAS. • Correlacionadas: Contienen referencia a la consulta externa. No pueden evaluarse separadamente. • No Correlacionadas: (la mas habitual) No Contienen referencia a la consulta externa. Pueden ser evaluadas por separado.	Operadores para subconsultas de tipo Tabla ALL en una comparación con una subconsulta limita el conjunto de resultados a sólo aquellos registros en los que la comparación es verdadera para todos los valores. SOME y ANY son sinónimos. Devuelve TRUE si la comparación es verdadera para cualquiera de los valores en la columna. <i>Muestra la población y los países dónde la población sea mayor que la media de poblaciones de todos los continentes.</i> 1. Media de población de los continentes y su nombre. select continent, avg(population) as media from country group by continent; 2. La media de los continentes Select avg(population) from country group by continent; 3. la subconsulta correcta SELECT name, population from country WHERE population > ALL (select avg(population) from country group by continent);														
Ejemplo de consulta correlacionada * La consulta interna utiliza la consulta externa para evaluarse. <i>Población máxima de cada continente (poblado) y el país de la población máxima.</i> <pre>SELECT Continent, Name, Population FROM Country c WHERE Population = (SELECT MAX(Population) FROM Country c2 WHERE c.Continent=c2.Continent AND Population > 0);</pre>	Subconsultas de tipo tabla en la cláusula FROM En la cláusula FROM vimos que se podía poner un nombre de tabla o un nombre de consulta, pues en vez de poner un nombre de consulta se puede poner directamente la sentencia SELECT correspondiente a esa consulta encerrada entre paréntesis. *A las consultas de la cláusula FROM debemos añadirle un ALIAS, en otro caso da error. <i>Obtén el nombre, la región, el continente y el presidente del gobierno de los países del mundo que todavía no se han independizado</i> <pre>select name, region, continent, headofstate from (select * from country where indepyear isnull) as Independencia;</pre>														
Subconsultas de tipo tabla en la cláusula FROM En la cláusula FROM vimos que se podía poner un nombre de tabla o un nombre de consulta, pues en vez de poner un nombre de consulta se puede poner directamente la sentencia SELECT correspondiente a esa consulta encerrada entre paréntesis. *A las consultas de la cláusula FROM debemos añadirle un ALIAS, en otro caso da error. <i>Obtén el nombre, la región, el continente y el presidente del gobierno de los países del mundo que todavía no se han independizado</i> <pre>select name, region, continent, headofstate from (select * from country where indepyear isnull) as Independencia;</pre> • IN y NOT IN no utilizan operadores. Este operador devuelve TRUE si la subconsulta devuelve al menos un resultado igual al operando de la izquierda. IN es equivalente a = A Ej>Select name, continent from country where continent like 'Europe' AND code IN (SELECT countrycode from countrylanguage where Language like 'Spanish'); --4 países • Otro IN es utilizarlo para sustituir un where largo. • NOT IN es equivalente a <> ALL.	Select países del continente europeo donde se habla español. <pre>SELECT CountryCod FROM countrylanguage WHERE lengua = 'Spa'; SELECT name FROM country WHERE continent='Europe'; SELECT name FROM country WHERE continent='Europe' and code = ANY (SELECT CountryCod FROM countrylanguage WHERE lengua='Spa'); * SOME muestra el mismo resultado. = ALL no muestra ningún resultado. <> ANY muestra en los que no se habla español</pre> EXISTS devuelve true si las filas se encuentran en la subconsulta. NOT EXISTS da false si no se encuentran las columnas en la subconsulta. <i>Usa EXISTS obtén los países de Europa donde se habla español</i> <pre>select name, code from country where continent like 'europe' and EXISTS (select * from countrylanguage where country.code=countrycode and language like 'spanish');</pre>														

ALTER TABLE le permite cambiar la estructura de una tabla existente. • Puede añadir o borrar columnas. • Crear o destruir índices. • Cambiar el tipo de columnas existentes. • Renombrar columnas • Renombrar la misma tabla. • Puede cambiar el comentario de la tabla y su tipo.	ALTER: Permite modificar la estructura de una tabla u objeto. Se pueden agregar/quitar campos a una tabla, modificar el tipo de un campo, agregar/quitar índices a una tabla, modificar un trigger, etc.
Sentencia ALTER TABLE <pre>ALTER TABLE <i>tbl_name</i> <i>alter_specification</i> [, <i>alter_specification</i>] ... <i>alter_specification</i>: ADD [COLUMN] column_definition [FIRST AFTER col_name]</pre>	RENAME Renombramos una tabla • ALTER TABLE t1 rename t2;
AÑADIR INDICE • ALTER TABLE t1 ADD primary key (campoA), ADD INDEX (b); • show index from t1; BORRAMOS EL INDICE b • ALTER TABLE t1 DROP INDEX b; BORRAMOS LA PRIMARY KEY, SIN PONER EL NOMBRE DEL CAMPO • ALTER TABLE t1 DROP PRIMARY KEY;	CHANGE Renombramos un campo. Nos obliga a poner todo lo que lleve el campo (el tipo, not null, etc) • ALTER TABLE t1 CHANGE a campoA int; Cambiamos el tipo de campo pero no el nombre. • ALTER TABLE t1 CHANGE b b BIGINT NOT NULL;
Restricciones de las vistas • La sentencia SELECT no puede : ○ Contener una subconsulta en su cláusula FROM. ○ Hacer referencia a variables del sistema o de usuario. ○ Hacer referencia a parámetros de sentencia preparados (PREPARE). • La definición no puede hacer referencia a una tabla TEMPORARY, y tampoco se puede crear una vista TEMPORARY. • Las tablas mencionadas en la definición de la vista deben existir siempre. • Cualquier tabla o vista referenciada por la definición debe existir. Sin embargo, <i>es posible que después de crear una vista, se elimine alguna tabla o vista a la que se hace referencia</i>. • En la definición de una vista está permitido ORDER BY, pero es ignorado si se seleccionan columnas de una vista que tiene su propio ORDER BY. • Si la definición de una vista incluye una cláusula LIMIT, y se hace una selección desde la vista utilizando una sentencia que tiene su propia cláusula LIMIT, no está definido cuál se aplicará. • El mismo principio se extiende a otras opciones como ALL, DISTINCT.	MODIFY Cambiamos el tipo de dato de una columna • ALTER TABLE t1 MODIFY b INT NULL; ADD AÑADIR CAMPOS. por defecto se añaden al final • ALTER TABLE t1 ADD nombre varchar(5) not null; AÑADIR CAMPOS AL PRINCIPIO -FIRST • ALTER TABLE t1 ADD dni varchar(9) not null FIRST; AÑADIR CAMPOS EN MEDIO- AFTER - <i>después del que sea, no existe before</i> • ALTER TABLE t1 ADD apellido varchar(10) after d; DROP BORRAR CAMPOS. • ALTER TABLE t1 DROP apellido; Se puede añadir varios drop para borrar varios a la vez e incluso añadir a la vez, separado por comas: • ALTER TABLE t1 DROP dni, DROP nombre, ADD provincia varchar(20);
Eliminar Vistas (Views) <pre>DROP VIEW [IF EXISTS] view_name [, view_name] .. ; mysql> DROP VIEW IF EXISTS v1, v2; Query OK, 0 rows affected, 1 warning (0.00 sec) mysql> SHOW WARNINGS;</pre>	Las tablas y las vistas comparten el mismo espacio de nombres en la base de datos, por eso, una base de datos no puede contener una tabla y una vista con el mismo nombre. Restricciones de la cláusula SELECT • Las vistas no pueden tener nombres de columnas duplicados. • Por defecto, los nombres de las columnas devueltos por la sentencia SELECT se usan para las columnas de la vista. • Para dar explícitamente un nombre a las columnas de la vista utilice la cláusula columnas para indicar una lista de nombres separados con comas. La cantidad de nombres indicados en columnas debe ser igual a la cantidad de columnas devueltas por la sentencia SELECT. • Las columnas devueltas por la sentencia SELECT pueden ser simples referencias a columnas de la tabla, pero también pueden ser expresiones conteniendo: funciones, constantes, operadores, etc. • Los nombres de tablas o vistas sin calificar en la sentencia SELECT se interpretan como pertenecientes a la base de datos actual. Una vista puede hacer referencia a tablas o vistas en otras bases de datos precediendo el nombre de la tabla o vista con el nombre de la base de datos apropiada. • Las vistas pueden crearse a partir de varios tipos de sentencias SELECT. Pueden hacer referencia a tablas o a otras vistas. Pueden usar combinaciones, UNION, y subconsultas. VISTAS actualizables La principal condición para la actualización es que debe haber una relación uno a uno entre las filas de la vista y los registros de la tabla base, y que las columnas de la vista que se actualizarán deben ser definidos como una la tabla de referencias simples, sin expresiones.

VACIA UNA TABLA con TRUNCATE

- Create table copy_city select * from city; - **copiamos una tabla**
- select * from copy_city; -- **la vemos**
- truncate copy_city; **vacía la tabla como el Delete ***
- select * from copy_city; -- **la vemos vacía**

- **UNION** (los registros que se repitan no se incluyen) une las tablas, pero tienen que tener la misma estructura
- **UNION** une los registros diferentes, no repite los que tienen la misma clave
- **UNION** une **2 TABLAS**, pero aquellos **registros** que sean **diferentes**

- **UNION ALL** (une toda la tabla con todos los registros)

- **UNION ALL**, las une aunque se repitan los registros

- **UNION ALL**, **une todos** los registros **estén o no repetidos**

- Las tablas tienen que tener la **misma estructura**

-- Todos los de la primera tabla más los distintos de la segunda.

Los campos han de llamarse igual:

Select * from coches **UNION Select** * from coches2;

Da error, en coches pido 2 y en coches2 estoy pidiendo todos:

Select nombre, descripción from coches **UNION**

select * from coches2;

Puedo unir de diferentes bases de datos:

Select nombre from coches **UNION**

SELECT nombre from colegio4.alumnos;

- **primero saca todos los nombres de coche y luego los de alumnos, los campos se llaman igual puedo unir tablas con diferente formato (PERO MOSTRAR SOLO EL CAMPO O CAMPOS COMUNES)**

-- **UNION ALL** une todos los registros de las 2 tablas estén repetidos o no

Select * from coches **UNION ALL**

select * from coches2; -- 11 registros

-- **pueden unirse 2 o más tablas por sus registros de igual nombre de campo**

Select nombre from coches **UNION ALL**

select nombre from coches2 **UNION ALL**

select nombre from colegio4.alumnos; -- 22 registros